

Logic Synthesis and Verification

PA3 Reinforcement Learning for Logic Synthesis Library

Tuning for Technology Mapping

B11107051 李品翰

Assignment 1 (30%): Environment setup and baseline replication

```
tion-PA3-MapTune$ for b in s13207 c2670 b20_1;do for l in 7nm gf180mcu_ff_125C gf180mcu_tt_025C nan45 sky130;do pa3_h
andin/script/mapping.py $l.lib $l.genlib benchmarks/$b.bench;done;done
[s13207.bench][7nm.genlib] Delay: 645.03 ps | Area: 1949.99
[s13207.bench][gf180mcu_ff_125C.genlib] Delay: 14748.01 ps | Area: 33061.91
[s13207.bench][gf180mcu_tt_025C.genlib] Delay: 5901.33 ps | Area: 33217.77
[s13207.bench][nan45.genlib] Delay: 703.66 ps | Area: 1920.52
[s13207.bench][sky130.genlib] Delay: 3228.11 ps | Area: 10493.81
[c2670.bench][7nm.genlib] Delay: 155.37 ps | Area: 461.66
[c2670.bench][gf180mcu_ff_125C.genlib] Delay: 8680.05 ps | Area: 7371.48
[c2670.bench][gf180mcu_tt_025C.genlib] Delay: 3442.8 ps | Area: 7580.03
[c2670.bench][nan45.genlib] Delay: 441.04 ps | Area: 493.96
[c2670.bench][sky130.genlib] Delay: 1907.89 ps | Area: 2308.46
[b20_1.bench][7nm.genlib] Delay: 656.76 ps | Area: 5034.18
[b20_1.bench][gf180mcu_ff_125C.genlib] Delay: 37376.29 ps | Area: 103310.5
[b20_1.bench][gf180mcu_tt_025C.genlib] Delay: 15583.81 ps | Area: 102943.91
[b20_1.bench][nan45.genlib] Delay: 1573.15 ps | Area: 6641.49
[b20_1.bench][sky130.genlib] Delay: 6660.12 ps | Area: 33651.02
```

Table 1: Baseline Technology Mapping Results

Benchmark	Library	Delay (ps)	Area (μm^2)
s13207	7nm	645.03	1949.99
	gf180mcu_ff_125C	14748.01	33061.91
	gf180mcu_tt_025C	5901.33	33217.77
	nan45	703.66	1920.52
	sky130	3228.11	10493.81
c2670	7nm	155.37	461.66
	gf180mcu_ff_125C	8680.05	7371.48
	gf180mcu_tt_025C	3442.80	7580.03
	nan45	441.04	493.96
	sky130	1907.89	2308.46
b20_1	7nm	656.76	5034.18
	gf180mcu_ff_125C	37376.29	103310.50
	gf180mcu_tt_025C	15583.81	102943.91
	nan45	1573.15	6641.49
	sky130	6660.12	33651.02

Assignment 2 (30%): Randomly sampling replication

從 Figure 1 可以看出，即便只取 75 ~ 150 個 cells (藍色部分)，仍然可能得到相當不錯的解，證明了 partial library 仍有機會贏過 full library。

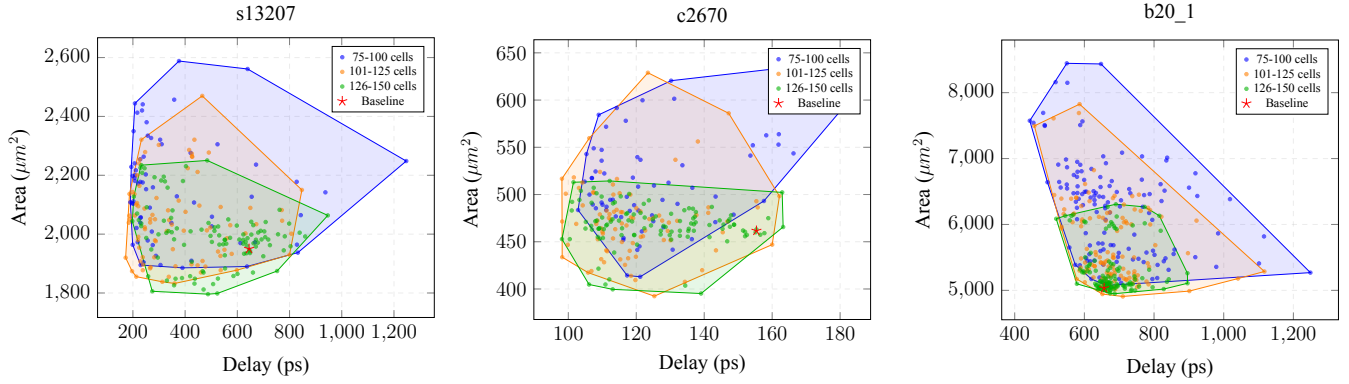


Figure 1: Area-Delay scatter plot for randomly sample of ASAP7 (7nm.lib)

Assignment 3 (40%): Standing on the shoulders of MapTune

基於 MapTune 的結果 $S_{MapTune}$ ，我透過執行數輪 First-Choice Local Search，也就是不停尋找當前最佳解的鄰居，一一嘗試是否能找到更好的解。如 Algorithm 1 所示，鄰居即為將當前選擇 S_{curr} 加入或移除一個 cell 之後的新集合，而為了確保 $|S_{new}| \leq |S_{MapTune}|$ ，若當前選擇數量已經達到上限，將不允許加入 cell，只允許移除。若該輪沒有找到更好的解，則選擇本輪相對最好的解作為下一輪的初始解。若連續 2 輪沒有找到更好的解則結束，設定為 2 輪的原因是：令當前最佳解為 S_{best} ，若本輪沒找到更好的解，則會選擇本輪相對好的解 S'_{best} 進入第二輪，由於 S_{best} 與 S'_{best} 互為鄰居，若第二輪也沒有找到更好的解，則必定會選擇 S_{best} 進入第三輪，如此將產生不必要的重複嘗試，因此 2 輪就足夠了。

Algorithm 1: First-Choice Local Search for Library Tuning

Require: $\mathcal{F}$ (set of all available cells), $S_{MapTune}$ (initial selected cells from MapTune)

- 1: $S_{best} \leftarrow S_{MapTune}, S_{curr} \leftarrow S_{MapTune}$
- 2: $R_{best} \leftarrow \text{CalculateReward}(S_{MapTune})$
- 3: $P_{no_improv} \leftarrow 0$ ▷ Counter for consecutive non-improving rounds
- 4: **while** $P_{no_improv} < 2$ **do**
- 5: $R_{round} \leftarrow -\infty, S_{round} \leftarrow \emptyset$
- 6: $found_better \leftarrow \text{false}$
- 7: **for each** $c \in \mathcal{F}$ **do**
- 8: **if** $c \in S_{curr}$ **then**
- 9: $S_{sel} \leftarrow S_{curr} \setminus \{c\}$ ▷ Remove currently selected cell
- 10: **else**
- 11: **if** $|S_{curr}| \geq |S_{MapTune}|$ **then**
- 12: **continue** ▷ Skip adding if budget is reached
- 13: **end if**
- 14: $S_{sel} \leftarrow S_{curr} \cup \{c\}$ ▷ Add unselected cell
- 15: **end if**

```

16:    $r \leftarrow \text{CalculateReward}(S_{sel})$ 
17:   if  $r > R_{round}$  then
18:      $R_{round} \leftarrow r, S_{round} \leftarrow S_{sel}$ 
19:   end if
20:   if  $r > R_{best}$  then
21:      $R_{best} \leftarrow r, S_{best} \leftarrow S_{sel}$ 
22:      $S_{curr} \leftarrow S_{sel}$ 
23:      $P_{no\_improv} \leftarrow 0$ 
24:      $found\_better \leftarrow \text{true}$ 
25:     break ▷ Accept the first better solution immediately
26:   end if
27: end for
28: if not  $found\_better$  then ▷ Fallback to the best candidate in this round
29:    $S_{curr} \leftarrow S_{round}$ 
30:    $P_{no\_improv} \leftarrow P_{no\_improv} + 1$ 
31: end if
32: end while
33: return  $S_{best}, R_{best}$ 

```

以 c2670.bench + nan45.genlib 選擇 54 個 cells 為例，如下方執行結果所示，在 MapTune 完成 100 個 iteration，得到約 -0.873 的 reward 之後，我的程式成功找到了 7 次更好的解，最終將 reward 提升到了約 -0.858。其中 success 代表該輪找到了更好的解，fail 則否。

```

$ python batched_MAB_EP.py 54 benchmarks/c2670.bench nan45.genlib
select 54 cells from 82 cells
keep 12 cells
Baseline Delay: 441.04
Baseline Area: 493.96
Batch iteration: 0
Current best reward: -0.9626917908272263
Current best result: (440.53, 458.32)
Batch iteration: 1
Current best reward: -0.9626917908272263
Current best result: (440.53, 458.32)
Batch iteration: 2
Current best reward: -0.8862500973818875
Current best result: (366.33, 467.1)

...

Batch iteration: 99
Current best reward: -0.8726437910806986
Current best result: (359.47, 461.51)
Best Delay: 359.47
Best Area: 461.51
Best Reward: -0.8726437910806986
Total time: 111.19865918159485
  1: success, best reward: -0.8692, delay: 355.85, area: 462.57, cells count: 53
  2: success, best reward: -0.8671, delay: 359.29, area: 455.92, cells count: 54
  3: success, best reward: -0.8664, delay: 354.78, area: 460.98, cells count: 53
  4: success, best reward: -0.8651, delay: 353.51, area: 461.24, cells count: 52
  5: success, best reward: -0.8602, delay: 349.11, area: 461.78, cells count: 53
  6: success, best reward: -0.8600, delay: 348.12, area: 462.84, cells count: 52

```

```

7: success, best reward: -0.8584, delay: 350.25, area: 458.32, cells count: 51
8: fail, round best reward: -0.8584, delay: 350.25, area: 458.32, cells count: 52
9: fail, round best reward: -0.8584, delay: 350.25, area: 458.32, cells count: 51
Best Reward: -0.8583977999741299
Best Delay: 350.25
Best Area: 458.32
Best Cells Count: 51
Total time: 149.6730215549469

```

Table 2 是對 s13207、c2670、b20_1 分別使用 nan45、sky130 在 budgets 為 30%、50%、70% 時，MapTune (batched_MAB_EP) 與我的方法的結果比較，可以發現我的方法成功達成了 $S_{new} \leq S_{MapTune}$ 且 $ADP_{new} < ADP_{MapTune}$ 的目標。

然而從執行時間可以發現，此方法仍然不太穩定，有些 case 比 MapTune 快上許多，但有些卻花了 MapTune 3 倍以上的時間執行。且 Local Search 並不能保證找到更優解，有可能在一開始就連續 2 輪失敗導致結束，因此未來或許可以嘗試加入了溫度的 Simulated Annealing，使其跳脫 Local Maximum 以找到更好的解。

Table 2: Performance Comparison between MapTune and Proposed Local Search

Benchmark	Library	Budget	MapTune					Proposed Method				
			Delay (ps)	Area (μm^2)	Reward	$ S_{MapTune} $	Time(s)	Delay (ps)	Area (μm^2)	Reward	$ S_{new} $	Time(s)
b20_1	nan45	30%	1223.85	7726.50	-0.9513	16	532.91	1162.30	7399.85	-0.9073	16	197.27
		50%	1434.47	7103.26	-0.9875	35	981.76	1400.69	6910.68	-0.9625	34	223.59
		70%	1402.04	6978.24	-0.9677	54	488.13	1336.58	7014.15	-0.9473	54	236.90
b20_1	sky130	30%	6392.30	36999.23	-1.0273	85	659.37	5735.84	36318.58	-0.9641	85	1099.80
		50%	6286.38	35713.00	-1.0009	154	679.09	5962.02	35630.42	-0.9736	154	1021.78
		70%	6147.25	34779.61	-0.9767	222	713.05	5611.83	34501.84	-0.9295	221	1478.92
c2670	nan45	30%	365.24	513.38	-0.9277	16	110.36	352.58	444.49	-0.8482	16	46.70
		50%	367.25	458.05	-0.8787	35	112.98	365.83	450.07	-0.8694	35	20.00
		70%	359.47	461.51	-0.8726	54	111.20	350.25	458.32	-0.8584	51	38.47
c2670	sky130	30%	1709.33	2448.60	-0.9748	85	184.36	1446.96	2398.55	-0.8877	84	196.47
		50%	1751.06	2253.41	-0.9465	154	186.22	1421.07	2223.38	-0.8470	154	359.68
		70%	1781.76	2312.22	-0.9672	222	206.09	1512.49	2232.14	-0.8755	221	287.58
s13207	nan45	30%	576.02	2220.04	-0.9728	16	175.45	522.57	2107.25	-0.9027	16	111.30
		50%	593.88	2033.84	-0.9454	35	175.74	558.39	2006.44	-0.9105	34	77.79
		70%	578.15	1992.34	-0.9232	54	164.28	551.34	1986.49	-0.9002	53	47.54
s13207	sky130	30%	2874.09	11091.89	-0.9701	85	262.23	2594.86	11065.61	-0.9207	83	736.86
		50%	2753.67	10625.19	-0.9294	154	276.75	2536.92	10721.53	-0.8961	153	739.71
		70%	2705.81	10854.16	-0.9311	222	279.45	2616.68	10866.67	-0.9162	213	969.61

Conclusion

本次作業透過 MapTune 框架探討了 partial library 對於邏輯合成中 PPA 的影響。實驗結果不僅驗證了縮減後的元件庫能帶來更佳的 Area-Delay Product (ADP)，我所嘗試的 First-Choice Local Search 演算法也成功在各個 benchmark 與 budget 限制下，穩定超越了原本 MapTune 的基準測試結果，同時嚴格滿足了 $|S_{new}| \leq |S_{MapTune}|$ 的條件限制。

雖然 Greedy 策略在執行時間上存在較大波動，且容易陷入局部最佳解，但整體而言，本實驗充分展示了自動化 Library Tuning 在 EDA 領域的巨大潛力。未來若能結合 Simulated Annealing 等更進階的啟發式演算法，預期能進一步提升優化穩定度與探索空間。

Source Code

完整實驗用到的程式碼可以在 <https://github.com/co0okie/NTUST-Logic-Synthesis-and-Verification-PA3-MapTune> 找到。